

310 我们的程序已经使用了很多 IO 库设施了。我们在 1.2 节（第 5 页）已经介绍了大部分 IO 库设施：

- `istream`（输入流）类型，提供输入操作。
- `ostream`（输出流）类型，提供输出操作。
- `cin`，一个 `istream` 对象，从标准输入读取数据。
- `cout`，一个 `ostream` 对象，向标准输出写入数据。
- `cerr`，一个 `ostream` 对象，通常用于输出程序错误消息，写入到标准错误。
- `>>` 运算符，用来从一个 `istream` 对象读取输入数据。
- `<<` 运算符，用来向一个 `ostream` 对象写入输出数据。
- `getline` 函数（参见 3.3.2 节，第 78 页），从一个给定的 `istream` 读取一行数据，存入一个给定的 `string` 对象中。



8.1 IO 类

到目前为止，我们已经使用过的 IO 类型和对象都是操纵 `char` 数据的。默认情况下，这些对象都是关联到用户的控制台窗口的。当然，我们不能限制实际应用程序仅从控制台窗口进行 IO 操作，应用程序常常需要读写命名文件。而且，使用 IO 操作处理 `string` 中的字符会很方便。此外，应用程序还可能读写需要宽字符支持的语言。

为了支持这些不同种类的 IO 处理操作，在 `istream` 和 `ostream` 之外，标准库还定义了一些其他 IO 类型，我们之前都已经使用过了。表 8.1 列出了这些类型，分别定义在三个独立的头文件中：`iostream` 定义了用于读写流的基本类型，`fstream` 定义了读写命名文件的类型，`sstream` 定义了读写内存 `string` 对象的类型。

表 8.1: IO 库类型和头文件	
头文件	类型
iostream	<code>istream</code> , <code>wistream</code> 从流读取数据
	<code>ostream</code> , <code>wostream</code> 向流写入数据
	<code>iostream</code> , <code>wiostream</code> 读写流
fstream	<code>ifstream</code> , <code>wifstream</code> 从文件读取数据
	<code>ofstream</code> , <code>wofstream</code> 向文件写入数据
	<code>fstream</code> , <code>wfstream</code> 读写文件
sstream	<code>istringstream</code> , <code>wistringstream</code> 从 <code>string</code> 读取数据
	<code>ostringstream</code> , <code>wostringstream</code> 向 <code>string</code> 写入数据
	<code>stringstream</code> , <code>wstringstream</code> 读写 <code>string</code>

311

为了支持使用宽字符的语言，标准库定义了一组类型和对象来操纵 `wchar_t` 类型的数据（参见 2.1.1 节，第 30 页）。宽字符版本的类型和函数的名字以一个 `w` 开始。例如，`wcin`、`wcout` 和 `wcerr` 是分别对应 `cin`、`cout` 和 `cerr` 的宽字符版对象。宽字符版本的类型和对象与其对应的普通 `char` 版本的类型定义在同一个头文件中。例如，头文件 `fstream` 定义了 `ifstream` 和 `wifstream` 类型。

IO 类型间的关系

概念上，设备类型和字符大小都不会影响我们要执行的 IO 操作。例如，我们可以用 >> 读取数据，而不用管是从一个控制台窗口，一个磁盘文件，还是一个 string 读取。类似的，我们也不用管读取的字符能存入一个 char 对象内，还是需要一个 wchar_t 对象来存储。

标准库使我们能忽略这些不同类型的流之间的差异，这是通过继承机制（inheritance）实现的。利用模板（参见 3.3 节，第 87 页），我们可以使用具有继承关系的类，而不必了解继承机制如何工作的细节。我们将在第 15 章和 18.3 节（第 710 页）介绍 C++ 是如何支持继承机制的。

简单地说，继承机制使我们声明一个特定的类继承自另一个类。我们通常可以将一个派生类（继承类）对象当作其基类（所继承的类）对象来使用。

类型 ifstream 和 istringstream 都继承自 istream。因此，我们可以像使用 istream 对象一样来使用 ifstream 和 istringstream 对象。也就是说，我们是如何使用 cin 的，就可以同样地使用这些类型的对象。例如，可以对一个 ifstream 或 istringstream 对象调用 getline，也可以使用 >> 从一个 ifstream 或 istringstream 对象中读取数据。类似的，类型 ofstream 和 ostream 都继承自 ostream。因此，我们是如何使用 cout 的，就可以同样地使用这些类型的对象。

 本节剩下部分所介绍的标准库流特性都可以无差别地应用于普通流、文件流和 string 流，以及 char 或宽字符流版本。

8.1.1 IO 对象无拷贝或赋值



如我们在 7.1.3 节（第 234 页）所见，我们不能拷贝或对 IO 对象赋值：

```
ofstream out1, out2;
out1 = out2; // 错误：不能对流对象赋值
ofstream print(outstream); // 错误：不能初始化 ofstream 参数
out2 = print(out2); // 错误：不能拷贝流对象
```

由于不能拷贝 IO 对象，因此我们也不能将形参或返回类型设置为流类型（参见 6.2.1 节，第 188 页）。进行 IO 操作的函数通常以引用方式传递和返回流。读写一个 IO 对象会改变其状态，因此传递和返回的引用不能是 const 的。

8.1.2 条件状态

IO 操作一个与生俱来的问题是可能发生错误。一些错误是可恢复的，而其他错误则发生在系统深处，已经超出了应用程序可以修正的范围。表 8.2 列出了 IO 类所定义的一些函数和标志，可以帮助我们访问和操纵流的条件状态（condition state）。

表 8.2: IO 库条件状态	
<code>strm::iostate</code>	<code>strm</code> 是一种 IO 类型，在表 8.1（第 278 页）中已列出。 <code>iostate</code> 是一种机器相关的类型，提供了表达条件状态的完整功能
<code>strm::badbit</code>	<code>strm::badbit</code> 用来指出流已崩溃
<code>strm::failbit</code>	<code>strm::failbit</code> 用来指出一个 IO 操作失败了

续表

<code>strm::eofbit</code>	<code>strm::eofbit</code> 用来指出流到达了文件结束
<code>strm::goodbit</code>	<code>strm::goodbit</code> 用来指出流未处于错误状态。此值保证为零
<code>s.eof()</code>	若流 <code>s</code> 的 <code>eofbit</code> 置位, 则返回 <code>true</code>
<code>s.fail()</code>	若流 <code>s</code> 的 <code>failbit</code> 或 <code>badbit</code> 置位, 则返回 <code>true</code>
<code>s.bad()</code>	若流 <code>s</code> 的 <code>badbit</code> 置位, 则返回 <code>true</code>
<code>s.good()</code>	若流 <code>s</code> 处于有效状态, 则返回 <code>true</code>
<code>s.clear()</code>	将流 <code>s</code> 中所有条件状态位复位, 将流的状态设置为有效。返回 <code>void</code>
<code>s.clear(flags)</code>	根据给定的 <code>flags</code> 标志位, 将流 <code>s</code> 中对应条件状态位复位。 <code>flags</code> 的类型为 <code>strm::iostate</code> 。返回 <code>void</code>
<code>s.setstate(flags)</code>	根据给定的 <code>flags</code> 标志位, 将流 <code>s</code> 中对应条件状态位置位。 <code>flags</code> 的类型为 <code>strm::iostate</code> 。返回 <code>void</code>
<code>s.rdstate()</code>	返回流 <code>s</code> 的当前条件状态, 返回值类型为 <code>strm::iostate</code>

下面是一个 IO 错误的例子:

```
int ival;
cin >> ival;
```

如果我们在标准输入上键入 `Boo`, 读操作就会失败。代码中的输入运算符期待读取一个 `int`, 但却得到了一个字符 `B`。这样, `cin` 会进入错误状态。类似的, 如果我们输入一个文件结束标识, `cin` 也会进入错误状态。

一个流一旦发生错误, 其上后续的 IO 操作都会失败。只有当一个流处于无错状态时, 我们才可以从它读取数据, 向它写入数据。由于流可能处于错误状态, 因此代码通常应该在使用一个流之前检查它是否处于良好状态。确定一个流对象的状态的最简单的方法是将它当作一个条件来使用:

```
while (cin >> word)
    // ok: 读操作成功.....
```

`while` 循环检查 `>>` 表达式返回的流的状态。如果输入操作成功, 流保持有效状态, 则条件为真。

查询流的状态

将流作为条件使用, 只能告诉我们流是否有效, 而无法告诉我们具体发生了什么。有时我们也需要知道流为什么失败。例如, 在键入文件结束标识后我们的应对措施, 可能与遇到一个 IO 设备错误的处理方式是不同的。

IO 库定义了一个与机器无关的 `iostate` 类型, 它提供了表达流状态的完整功能。这个类型应作为一个位集合来使用, 使用方式与我们在 4.8 节中 (第 137 页) 使用 `quiz1` 的方式一样。IO 库定义了 4 个 `iostate` 类型的 `constexpr` 值 (参见 2.4.4 节, 第 58 页), 表示特定的位模式。这些值用来表示特定类型的 IO 条件, 可以与位运算符 (参见 4.8 节, 第 137 页) 一起使用来一次性检测或设置多个标志位。

`badbit` 表示系统级错误, 如不可恢复的读写错误。通常情况下, 一旦 `badbit` 被置位, 流就无法再使用了。在发生可恢复错误后, `failbit` 被置位, 如期望读取数值却读出一个字符等错误。这种问题通常是可以修正的, 流还可以继续使用。如果到达文件结束位置, `eofbit` 和 `failbit` 都会被置位。`goodbit` 的值为 0, 表示流未发生错误。如果 `badbit`、`failbit` 和 `eofbit` 任一个被置位, 则检测流状态的条件会失败。

标准库还定义了一组函数来查询这些标志位的状态。操作 `good` 在所有错误位均未置位的情况下返回 `true`，而 `bad`、`fail` 和 `eof` 则在对应错误位被置位时返回 `true`。此外，在 `badbit` 被置位时，`fail` 也会返回 `true`。这意味着，使用 `good` 或 `fail` 是确定流的总体状态的正确方法。实际上，我们将流当作条件使用的代码就等价于 `!fail()`。而 `eof` 和 `bad` 操作只能表示特定的错误。

313

管理条件状态

流对象的 `rdstate` 成员返回一个 `iostate` 值，对应流的当前状态。`setstate` 操作将给定条件位置位，表示发生了对应错误。`clear` 成员是一个重载的成员（参见 6.4 节，第 206 页）：它有一个不接受参数的版本，而另一个版本接受一个 `iostate` 类型的参数。

`clear` 不接受参数的版本清除（复位）所有错误标志位。执行 `clear()` 后，调用 `good` 会返回 `true`。我们可以这样使用这些成员：

```
// 记住 cin 的当前状态
auto old_state = cin.rdstate(); // 记住 cin 的当前状态
cin.clear();                    // 使 cin 有效
process_input(cin);             // 使用 cin
cin.setstate(old_state);        // 将 cin 置为原有状态
```

带参数的 `clear` 版本接受一个 `iostate` 值，表示流的新状态。为了复位单一的条件状态位，我们首先用 `rdstate` 读出当前条件状态，然后用位操作将所需位复位来生成新的状态。例如，下面的代码将 `failbit` 和 `badbit` 复位，但保持 `eofbit` 不变：

314

```
// 复位 failbit 和 badbit，保持其他标志位不变
cin.clear(cin.rdstate() & ~cin.failbit & ~cin.badbit);
```

8.1.2 节练习

练习 8.1: 编写函数，接受一个 `istream&` 参数，返回值类型也是 `istream&`。此函数须从给定流中读取数据，直至遇到文件结束标识时停止。它将读取的数据打印在标准输出上。完成这些操作后，在返回流之前，对流进行复位，使其处于有效状态。

练习 8.2: 测试函数，调用参数为 `cin`。

练习 8.3: 什么情况下，下面的 `while` 循环会终止？

```
while (cin >> i) /* ... */
```

8.1.3 管理输出缓冲

每个输出流都管理一个缓冲区，用来保存程序读写的数据。例如，如果执行下面的代码

```
os << "please enter a value: ";
```

文本串可能立即打印出来，但也有可能被操作系统保存在缓冲区中，随后再打印。有了缓冲机制，操作系统就可以将程序的多个输出操作组合成单一的系统级写操作。由于设备的写操作可能很耗时，允许操作系统将多个输出操作组合为单一的设备写操作可以带来很大的性能提升。

导致缓冲刷新（即，数据真正写到输出设备或文件）的原因有很多：

- 程序正常结束，作为 `main` 函数的 `return` 操作的一部分，缓冲刷新被执行。

- 缓冲区满时，需要刷新缓冲，而后新的数据才能继续写入缓冲区。
- 我们可以使用操纵符如 `endl`（参见 1.2 节，第 6 页）来显式刷新缓冲区。
- 在每个输出操作之后，我们可以用操纵符 `unitbuf` 设置流的内部状态，来清空缓冲区。默认情况下，对 `cerr` 是设置 `unitbuf` 的，因此写到 `cerr` 的内容都是立即刷新的。
- 一个输出流可能被关联到另一个流。在这种情况下，当读写被关联的流时，关联到的流的缓冲区会被刷新。例如，默认情况下，`cin` 和 `cerr` 都关联到 `cout`。因此，读 `cin` 或写 `cerr` 都会导致 `cout` 的缓冲区被刷新。

315 刷新输出缓冲区

我们已经使用过操纵符 `endl`，它完成换行并刷新缓冲区的工作。IO 库中还有两个类似的操纵符：`flush` 和 `ends`。`flush` 刷新缓冲区，但不输出任何额外的字符；`ends` 向缓冲区插入一个空字符，然后刷新缓冲区：

```
cout << "hi!" << endl;    // 输出 hi 和一个换行，然后刷新缓冲区
cout << "hi!" << flush;   // 输出 hi，然后刷新缓冲区，不附加任何额外字符
cout << "hi!" << ends;    // 输出 hi 和一个空字符，然后刷新缓冲区
```

unitbuf 操纵符

如果想在每次输出操作后都刷新缓冲区，我们可以使用 `unitbuf` 操纵符。它告诉流在接下来的每次写操作之后都进行一次 `flush` 操作。而 `nounitbuf` 操纵符则重置流，使其恢复使用正常的系统管理的缓冲区刷新机制：

```
cout << unitbuf;           // 所有输出操作后都会立即刷新缓冲区
// 任何输出都立即刷新，无缓冲
cout << nounitbuf;        // 回到正常的缓冲方式
```

警告：如果程序崩溃，输出缓冲区不会被刷新

如果程序异常终止，输出缓冲区是会被刷新的。当一个程序崩溃后，它所输出的数据很可能停留在输出缓冲区中等待打印。

当调试一个已经崩溃的程序时，需要确认那些你认为已经输出的数据确实已经刷新了。否则，可能将大量时间浪费在追踪代码为什么没有执行上，而实际上代码已经执行了，只是程序崩溃后缓冲区没有被刷新，输出数据被挂起没有打印而已。

关联输入和输出流

当一个输入流被关联到一个输出流时，任何试图从输入流读取数据的操作都会先刷新关联的输出流。标准库将 `cout` 和 `cin` 关联在一起，因此下面语句

```
cin >> ival;
```

导致 `cout` 的缓冲区被刷新。



交互式系统通常应该关联输入流和输出流。这意味着所有输出，包括用户提示信息，都会在读操作之前被打印出来。

`tie` 有两个重载的版本（参见 6.4 节，第 206 页）：一个版本不带参数，返回指向输

出流的指针。如果本对象当前关联到一个输出流，则返回的就是指向这个流的指针，如果对象未关联到流，则返回空指针。tie 的第二个版本接受一个指向 ostream 的指针，将自己关联到此 ostream。即，x.tie(&o) 将流 x 关联到输出流 o。

我们既可以将一个 istream 对象关联到另一个 ostream，也可以将一个 ostream 关联到另一个 ostream:

```
cin.tie(&cout);           // 仅仅是用来展示：标准库将 cin 和 cout 关联在一起
// old_tie 指向当前关联到 cin 的流（如果有的话）
ostream *old_tie = cin.tie(nullptr); // cin 不再与其他流关联
// 将 cin 与 cerr 关联；这不是一个好主意，因为 cin 应该关联到 cout
cin.tie(&cerr);           // 读取 cin 会刷新 cerr 而不是 cout
cin.tie(old_tie);         // 重建 cin 和 cout 间的正常关联
```

在这段代码中，为了将一个给定的流关联到一个新的输出流，我们将新流的指针传递给了 tie。为了彻底解开流的关联，我们传递了一个空指针。每个流同时最多关联到一个流，但多个流可以同时关联到同一个 ostream。

8.2 文件输入输出



头文件 fstream 定义了三个类型来支持文件 IO: ifstream 从一个给定文件读取数据，ofstream 向一个给定文件写入数据，以及 fstream 可以读写给定文件。在 17.5.3 节中（第 676 页）我们将介绍如何对同一个文件流既读又写。

这些类型提供的操作与我们之前已经使用过的对象 cin 和 cout 的操作一样。特别是，我们可以用 IO 运算符（<<和>>）来读写文件，可以用 getline（参见 3.2.2 节，第 79 页）从一个 ifstream 读取数据，包括 8.1 节中（第 278 页）介绍的内容也都适用于这些类型。

除了继承自 istream 类型的行为之外，fstream 中定义的类型还增加了一些新的成员来管理与流关联的文件。在表 8.3 中列出了这些操作，我们可以对 fstream、ifstream 和 ofstream 对象调用这些操作，但不能对其他 IO 类型调用这些操作。

表 8.3: fstream 特有的操作	
<i>fstream</i> fstrm;	创建一个未绑定的文件流。 <i>fstream</i> 是头文件 <i>fstream</i> 中定义的一个类型
<i>fstream</i> fstrm(s);	创建一个 <i>fstream</i> ，并打开名为 s 的文件。s 可以是 string 类型，或者是一个指向 C 风格字符串的指针（参见 3.5.4 节，第 109 页）。这些构造函数都是 explicit 的（参见 7.5.4 节，第 265 页）。默认的文件模式 mode 依赖于 <i>fstream</i> 的类型
<i>fstream</i> fstrm(s, mode);	与前一个构造函数类似，但按指定 mode 打开文件
fstrm.open(s)	打开名为 s 的文件，并将文件与 fstrm 绑定。s 可以是一个 string 或一个指向 C 风格字符串的指针。默认的文件 mode 依赖于 <i>fstream</i> 的类型。返回 void
fstrm.close()	关闭与 fstrm 绑定的文件。返回 void
fstrm.is_open()	返回一个 bool 值，指出与 fstrm 关联的文件是否成功打开且尚未关闭